知习 Pro 论文关键代码与注释说明

真实源码摘录、路径标注与论文写作说明

项目名称	知习 Pro
项目类型	Android 期末应用设计与实现
开发语言	Java + XML + SQLite
原始工程	D:\android project\ZhiXiPro
生成日期	2026 年 10 月 23 日

文档说明

这份文档用于期末报告写作和代码核对，不是 Android 工程本身。每段 Java 代码都由生成程序直接读取项目源文件后原样写入，没有手工改写代码内容。中文解释放在代码块之外，因此代码可以逐行与原文件比对。

论文用源码副本：D:\ZhiXiPro\src。下方“原始文件路径”指向真实开发工程，可方便直接在 Android Studio 中核对。

截图建议：在 Android Studio 打开真实工程，把 Project 窗口切换为 Project 视图；按本文给出的文件路径和行号打开代码，保留行号、文件标签和核心方法后截图。

代码引用总览

报告位置	功能模块	源码类别
第 3 章	数据库建表、外键与迁移	DatabaseHelper
第 4.1 节	密码哈希、记住密码、登录	PasswordUtil / AppPreferences / UserDao
第 4.2-4.3 节	打卡清单、自习室、ListView	TaskDao / StudyRoomDao / Adapter
第 4.4-4.5 节	智能选座、预约事务、冲突校验	SeatDao / Recommendation / ReservationDao
第 4.6 节	签到、签退、爽约与时间修复	CheckInRules / CheckInDao
第 4.7 节	专注计时和每日目标	FocusTimerState / FocusStateStore
第 5 章	RecyclerView 自学控件	ReservationAdapter
第 4.8 节	统计聚合、建议和自绘图表	StudyStatisticsDao / StudyBarChartView
第 4.9-4.10 节	主题、字号和管理端	ThemeManager / AdminSeatActivity

第 3 章 数据库设计

本项目使用 SQLiteOpenHelper 管理本地数据。正文建议展示建表、外键和迁移，不粘贴数据库类全部代码。

SQLite 外键开启

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\DatabaseHelper.java
范围	第 46 至 49 行
内容	DatabaseHelper.onConfigure

```
    public void onConfigure(SQLiteDatabase db) {
        super.onConfigure(db);
        db.setForeignKeyConstraintsEnabled(true);
    }
```

代码 1 SQLite 外键开启（原文件共 297 行）

代码说明：每次打开数据库连接时显式开启外键约束，保证预约、座位和签到记录之间的关联不是只写在表结构里。

论文写作提示：可以说明三张以上表之间确实存在外键约束，而不是把全部数据塞进一张表。

用户表建表语句

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\DatabaseHelper.java
范围	第 53 至 65 行
内容	users 表结构

```
        db.execSQL(
                "CREATE TABLE " + TABLE_USERS + " (" +
                        "user_id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                        "username TEXT NOT NULL COLLATE NOCASE UNIQUE, " +
                        "display_name TEXT NOT NULL, " +
                        "password_hash TEXT NOT NULL, " +
                        "password_salt TEXT NOT NULL, " +
                        "password_iterations INTEGER NOT NULL, " +
                        "auto_login_token_hash TEXT, " +
                        "created_at INTEGER NOT NULL, " +
                        "last_login_at INTEGER" +
                        ")"
        );
```

代码 2 用户表建表语句（原文件共 297 行）

代码说明：用户表保存用户名、昵称、密码摘要、独立盐、哈希迭代次数、最近登录时间和自动登录令牌摘要。

论文写作提示：结合数据库字段表说明用户名唯一、密码不存明文，以及登录时间用于账号记录。

预约表与外键

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\DatabaseHelper.java
范围	第 101 至 118 行
内容	reservations 表结构

```
        db.execSQL(
                "CREATE TABLE " + TABLE_RESERVATIONS + " (" +
                        "reservation_id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                        "user_id INTEGER NOT NULL, " +
                        "seat_id INTEGER NOT NULL, " +
                        "start_at INTEGER NOT NULL, " +
                        "end_at INTEGER NOT NULL, " +
                        "status TEXT NOT NULL DEFAULT 'BOOKED' " +
                        "CHECK(status IN ('BOOKED', 'CHECKED_IN', 'COMPLETED', " +
                        "'CANCELLED', 'NO_SHOW')), " +
                        "created_at INTEGER NOT NULL, " +
                        "CHECK(end_at > start_at), " +
                        "FOREIGN KEY(user_id) REFERENCES " + TABLE_USERS +
                        "(user_id) ON UPDATE CASCADE ON DELETE CASCADE, " +
                        "FOREIGN KEY(seat_id) REFERENCES " + TABLE_SEATS +
                        "(seat_id) ON UPDATE CASCADE ON DELETE RESTRICT" +
                        ")"
        );
```

代码 3 预约表与外键（原文件共 297 行）

代码说明：预约表通过 user_id 关联用户、seat_id 关联座位，并保存开始时间、结束时间和预约状态。结束时间必须晚于开始时间。

论文写作提示：这里是数据库设计的重点，可配 E-R 图和预约状态图。

数据库升级

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\DatabaseHelper.java
范围	第 284 至 296 行
内容	DatabaseHelper.onUpgrade

```
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        if (oldVersion < 2) {
            createStudyTasksTable(db);
            seedDemoRoomsIfEmpty(db);
        }
        if (oldVersion < 3) {
            db.execSQL(
                    "ALTER TABLE " + TABLE_STUDY_SESSIONS +
                            " ADD COLUMN session_token TEXT"
            );
        }
```

```
        createIndexes(db);
    }
```

代码 4 数据库升级（原文件共 297 行）

代码说明：升级过程中先补打卡表，再为学习记录增加唯一会话令牌，最后补索引，避免已有账号和学习记录丢失。

论文写作提示：说明版本 1 到版本 3 使用了增量迁移，体现数据兼容意识。

第 4.1 节 注册与登录

登录模块同时处理密码安全、记住密码、自动登录与最近登录时间。正文应重点说明密码哈希与数据库校验。

密码加盐哈希与验证

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\util\PasswordUtil.java
范围	第 31 至 69 行
内容	hashPassword 与 verifyPassword

```
    public static PasswordDigest hashPassword(String password) {
        byte[] salt = new byte[SALT_LENGTH_BYTES];
        SECURE_RANDOM.nextBytes(salt);
        char[] passwordChars = password.toCharArray();

        try {
            byte[] hash = derive(passwordChars, salt, ITERATIONS);
            return new PasswordDigest(encode(hash), encode(salt), ITERATIONS);
        } finally {
            java.util.Arrays.fill(passwordChars, '\0');
        }
    }

    public static boolean verifyPassword(
            String password,
            String expectedHash,
            String salt,
            int iterations
    ) {
        char[] passwordChars = password.toCharArray();
        byte[] actual = null;
        byte[] expected = null;

        try {
            actual = derive(passwordChars, decode(salt), iterations);
            expected = decode(expectedHash);
            return MessageDigest.isEqual(actual, expected);
        } catch (IllegalArgumentException exception) {
            return false;
        } finally {
            java.util.Arrays.fill(passwordChars, '\0');
            if (actual != null) {
                java.util.Arrays.fill(actual, (byte) 0);
            }
            if (expected != null) {
                java.util.Arrays.fill(expected, (byte) 0);
            }
        }
    }
```

代码说明：注册时用随机盐生成 PBKDF2 摘要，登录时重新计算并使用 MessageDigest.isEqual 做常量时间比较。数据库没有密码明文。

论文写作提示：这是账号安全部分的核心代码，建议配注册或登录截图。

记住密码的加密存储

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\util\AppPreferences.java
范围	第 53 至 102 行
内容	保存与读取加密密码

```
    public boolean saveRememberedCredentials(String username, String password) {
        try {
            SecretKey key = getOrCreateKey();
            Cipher cipher = Cipher.getInstance(TRANSFORMATION);
            cipher.init(Cipher.ENCRYPT_MODE, key);
            byte[] encrypted = cipher.doFinal(password.getBytes(StandardCharsets.UTF_8));

            preferences.edit()
                    .putString(KEY_USERNAME, username)
                    .putString(KEY_PASSWORD_CIPHER, Base64.encodeToString(
                            encrypted,
                            Base64.NO_WRAP
                    ))
                    .putString(KEY_PASSWORD_IV, Base64.encodeToString(
                            cipher.getIV(),
                            Base64.NO_WRAP
                    ))
                    .apply();
            return true;
        } catch (GeneralSecurityException | IllegalArgumentException exception) {
            clearRememberedPassword();
            return false;
        }
    }

    public String getRememberedPassword() {
        String encryptedValue = preferences.getString(KEY_PASSWORD_CIPHER, "");
        String ivValue = preferences.getString(KEY_PASSWORD_IV, "");
        if (encryptedValue.isEmpty() || ivValue.isEmpty()) {
            return null;
        }

        try {
            SecretKey key = getOrCreateKey();
            Cipher cipher = Cipher.getInstance(TRANSFORMATION);
            cipher.init(
                    Cipher.DECRYPT_MODE,
                    key,
                    new GCMParameterSpec(
                            GCM_TAG_BITS,
                            Base64.decode(ivValue, Base64.NO_WRAP)
                    )
            );
            byte[] plain = cipher.doFinal(Base64.decode(encryptedValue, Base64.NO_WRAP));
            return new String(plain, StandardCharsets.UTF_8);
        } catch (GeneralSecurityException | IllegalArgumentException exception) {
            clearRememberedPassword();
```

```
            return null;
        }
    }
```

代码 6 记住密码的加密存储（原文件共 184 行）

代码说明：密码经 Android Keystore 的 AES/GCM 加密后保存，IV 与密文分开保存。读取时再解密回填，不向 SharedPreferences 写明文。

论文写作提示：说明“记住密码”和“数据库密码哈希”分别采用不同的安全方案。

注册与登录数据库操作

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\UserDao.java
范围	第 32 至 105 行
内容	register 与 authenticate

```
    public User register(String username, String password, String requestedDisplayName)
            throws UserAlreadyExistsException {
        String normalizedUsername = username.trim();
        if (usernameExists(normalizedUsername)) {
            throw new UserAlreadyExistsException();
        }

        PasswordUtil.PasswordDigest digest = PasswordUtil.hashPassword(password);
        String displayName = requestedDisplayName == null
                ? ""
                : requestedDisplayName.trim();
        if (displayName.isEmpty()) {
            displayName = normalizedUsername;
        }

        ContentValues values = new ContentValues();
        values.put("username", normalizedUsername);
        values.put("display_name", displayName);
        values.put("password_hash", digest.getHash());
        values.put("password_salt", digest.getSalt());
        values.put("password_iterations", digest.getIterations());
        values.put("created_at", System.currentTimeMillis());

        try {
            long userId = databaseHelper
                    .getWritableDatabase()
                    .insertOrThrow(DatabaseHelper.TABLE_USERS, null, values);
            User user = findById(userId);
            if (user == null) {
                throw new IllegalStateException("User was inserted but cannot be queried");
            }
            return user;
        } catch (SQLiteConstraintException exception) {
            throw new UserAlreadyExistsException();
        }
    }

    public User authenticate(String username, String password) {
        SQLiteDatabase database = databaseHelper.getReadableDatabase();
        String[] columns = {
                "user_id",
                "username",
                "display_name",
                "password_hash",
```

```
                "password_salt",
                "password_iterations",
                "created_at",
                "last_login_at",
                "auto_login_token_hash"
        };

        try (Cursor cursor = database.query(
                DatabaseHelper.TABLE_USERS,
                columns,
                "username = ? COLLATE NOCASE",
                new String[]{username.trim()},
                null,
                null,
                null,
                "1"
        )) {
            if (!cursor.moveToFirst()) {
                return null;
            }

            boolean valid = PasswordUtil.verifyPassword(
                    password,
                    cursor.getString(cursor.getColumnIndexOrThrow("password_hash")),
                    cursor.getString(cursor.getColumnIndexOrThrow("password_salt")),
                    cursor.getInt(cursor.getColumnIndexOrThrow("password_iterations"))
            );
            return valid ? readUser(cursor) : null;
        }
    }
```

代码 7 注册与登录数据库操作（原文件共 214 行）

代码说明：注册时检查用户名是否重复并写入密码摘要；登录时按用户名查询盐、摘要和迭代次数，再验证密码并返回 User 对象。

论文写作提示：可与 users 表字段说明配合，展示完整的账号数据流。

登录业务与自动登录

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\activity\LoginActivity.java
范围	第 206 至 288 行
内容	performLogin

```
    private void performLogin() {
        usernameLayout.setError(null);
        passwordLayout.setError(null);

        String username = textOf(usernameInput);
        String password = textOf(passwordInput);

        if (TextUtils.isEmpty(username)) {
            usernameLayout.setError(getString(R.string.error_username_required));
            usernameInput.requestFocus();
            return;
        }
        if (TextUtils.isEmpty(password)) {
            passwordLayout.setError(getString(R.string.error_password_required));
            passwordInput.requestFocus();
            return;
        }
```

```
        boolean rememberPassword = rememberPasswordCheckbox.isChecked();
        boolean autoLogin = autoLoginCheckbox.isChecked();
        setAuthenticating(true, getString(R.string.login_working));

        authExecutor.execute(() -> {
            try {
                User user = userDao.authenticate(username, password);
                if (user == null) {
                    runOnUiThread(() -> {
                        if (!isFinishing() && !isDestroyed()) {
                            setAuthenticating(false, getString(R.string.action_enter_app));
                            showLoginFailure();
                        }
                    });
                    return;
                }

                boolean remembered = false;
                if (rememberPassword) {
                    remembered = preferences.saveRememberedCredentials(username, password);
                }

                String loginToken = null;
                String tokenHash = null;
                if (autoLogin && remembered) {
                    loginToken = PasswordUtil.generateLoginToken();
                    tokenHash = PasswordUtil.sha256(loginToken);
                    preferences.setAutoLoginToken(loginToken);
                } else {
                    preferences.clearAutoLogin();
                }

                userDao.updateLoginState(
                        user.getUserId(),
                        System.currentTimeMillis(),
                        tokenHash
                );
                if (!rememberPassword) {
                    preferences.clearRememberedCredentials();
                }

                boolean finalRemembered = remembered;
                runOnUiThread(() -> {
                    if (isFinishing() || isDestroyed()) {
                        return;
                    }
                    if (rememberPassword && !finalRemembered) {
                        Toast.makeText(
                                this,
                                R.string.remember_password_failed,
                                Toast.LENGTH_LONG
                        ).show();
                    }
                    openHome(user);
                });
            } catch (RuntimeException exception) {
                runOnUiThread(() -> {
                    if (!isFinishing() && !isDestroyed()) {
                        setAuthenticating(false, getString(R.string.action_enter_app));
                        showLoginFailure();
                    }
                });
            }
        });
    }
```

代码 8 登录业务与自动登录（原文件共 328 行）

代码说明：点击登录后将数据库验证放到后台线程，成功时更新最近登录时间、处理记忆密码和自动登录令牌，之后进入首页。

论文写作提示：说明 UI 线程不直接处理数据库与哈希密集操作，避免界面卡顿。

第 4.2 节 每日打卡清单

打卡清单补充了每日任务记录，既满足 SQLite CRUD，也提供完成勾选和首页进度联动。

任务 CRUD

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\TaskDao.java
范围	第 24 至 86 行
内容	查询、新增、完成、删除

```
public List<StudyTask> getTasksForDate(long userId, String taskDate) {
    List<StudyTask> tasks = new ArrayList<>();
    String sql = "SELECT task_id, user_id, task_date, title, is_completed, " +
            "completed_at, created_at FROM " + DatabaseHelper.TABLE_STUDY_TASKS +
            " WHERE user_id = ? AND task_date = ?" +
            " ORDER BY is_completed ASC, sort_order ASC, created_at DESC";
    try (Cursor cursor = databaseHelper.getReadableDatabase().rawQuery(
            sql,
            new String[]{String.valueOf(userId), taskDate}
    )) {
        while (cursor.moveToNext()) {
            tasks.add(new StudyTask(
                    cursor.getLong(0),
                    cursor.getLong(1),
                    cursor.getString(2),
                    cursor.getString(3),
                    cursor.getLong(6),
                    cursor.getInt(4) == 1,
                    cursor.isNull(5) ? 0 : cursor.getLong(5)
            ));
        }
    }
    return tasks;
}

public long addTask(long userId, String taskDate, String title) {
    ContentValues values = new ContentValues();
    values.put("user_id", userId);
    values.put("task_date", taskDate);
    values.put("title", title.trim());
    values.put("is_completed", 0);
    values.put("created_at", System.currentTimeMillis());
    values.put("sort_order", System.currentTimeMillis());
    return databaseHelper.getWritableDatabase().insertOrThrow(
            DatabaseHelper.TABLE_STUDY_TASKS,
            null,
            values
    );
}

public boolean setCompleted(long taskId, boolean completed) {
    ContentValues values = new ContentValues();
    values.put("is_completed", completed ? 1 : 0);
    if (completed) {
        values.put("completed_at", System.currentTimeMillis());
    } else {
```

```
            values.putNull("completed_at");
        }
        return databaseHelper.getWritableDatabase().update(
                DatabaseHelper.TABLE_STUDY_TASKS,
                values,
                "task_id = ?",
                new String[]{String.valueOf(taskId)}
        ) > 0;
    }

    public boolean deleteTask(long taskId) {
        return databaseHelper.getWritableDatabase().delete(
                DatabaseHelper.TABLE_STUDY_TASKS,
                "task_id = ?",
                new String[]{String.valueOf(taskId)}
        ) > 0;
    }
```

代码 9 任务 CRUD（原文件共 107 行）

代码说明：任务按用户和日期查询；完成状态更新时同步写入 completed_at；删除直接按任务主键执行。

论文写作提示：适合放进功能实现中的数据 CRUD 小节。

ListView 任务适配器

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\adapter\TaskAdapter.java
范围	第 60 至 94 行
内容	TaskAdapter.getView

```
    public View getView(int position, View convertView, ViewGroup parent) {
        ViewHolder holder;
        if (convertView == null) {
            convertView = inflater.inflate(R.layout.item_task, parent, false);
            holder = new ViewHolder(convertView);
            convertView.setTag(holder);
        } else {
            holder = (ViewHolder) convertView.getTag();
        }

        StudyTask task = getItem(position);
        holder.checkBox.setOnCheckedChangeListener(null);
        holder.checkBox.setChecked(task.isCompleted());
        holder.title.setText(task.getTitle());
        holder.state.setText(task.isCompleted()
                ? R.string.task_completed
                : R.string.task_pending);

        int flags = holder.title.getPaintFlags();
        if (task.isCompleted()) {
            flags |= Paint.STRIKE_THRU_TEXT_FLAG;
        } else {
            flags &= ~Paint.STRIKE_THRU_TEXT_FLAG;
        }
        holder.title.setPaintFlags(flags);
        holder.title.setAlpha(task.isCompleted() ? 0.55f : 1f);

        holder.checkBox.setOnCheckedChangeListener((button, checked) -> {
            if (task.isCompleted() != checked) {
                listener.onCompletedChanged(task, checked);
            }
        });
```

```
        holder.delete.setOnClickListener(view -> listener.onDeleteRequested(task));
        return convertView;
    }
```

代码 10 ListView 任务适配器（原文件共 109 行）

代码说明：列表复用 ViewHolder，重新绑定前先移除旧监听，避免 RecyclerView 类似复用场景中的错误回调；完成状态通过删除线体现。

论文写作提示：可作为 ListView 的辅助代码；RecyclerView 仍放在报告第 5 章单独讲解。

第 4.3 节 自习室与 ListView

自习室模块使用数据库做增删改查，并通过 ListView 展示数据库记录。

自习室查询与新增

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\StudyRoomDao.java
范围	第 38 至 103 行
内容	getRooms 与 insertRoom

```
    public List<StudyRoom> getRooms(String keyword) {
        List<StudyRoom> rooms = new ArrayList<>();
        String trimmedKeyword = keyword == null ? "" : keyword.trim();
        String like = "%" + trimmedKeyword + "%";
        String where = trimmedKeyword.isEmpty()
                ? ""
                : "WHERE r.name LIKE ? OR r.building LIKE ? OR r.location LIKE ? ";
        String order = "GROUP BY r.room_id " +
                "ORDER BY CASE WHEN r.status = 'OPEN' THEN 0 ELSE 1 END, " +
                "r.building COLLATE NOCASE, r.floor, r.name";

        String[] args = trimmedKeyword.isEmpty()
                ? null
                : new String[]{like, like, like};
        try (Cursor cursor = databaseHelper.getReadableDatabase().rawQuery(
                ROOM_SELECT + where + order,
                args
        )) {
            while (cursor.moveToNext()) {
                rooms.add(readRoom(cursor));
            }
        }
        return rooms;
    }

    public StudyRoom findById(long roomId) {
        try (Cursor cursor = databaseHelper.getReadableDatabase().rawQuery(
                ROOM_SELECT + "WHERE r.room_id = ? GROUP BY r.room_id",
                new String[]{String.valueOf(roomId)}
        )) {
            return cursor.moveToFirst() ? readRoom(cursor) : null;
        }
    }

    public long insertRoom(
            String name,
```

```
            String building,
            String floor,
            String location,
            String openTime,
            String closeTime
    ) {
        SQLiteDatabase database = databaseHelper.getWritableDatabase();
        database.beginTransaction();
        try {
            ContentValues values = new ContentValues();
            values.put("name", name.trim());
            values.put("building", building.trim());
            values.put("floor", floor.trim());
            values.put("location", location.trim());
            values.put("open_time", openTime);
            values.put("close_time", closeTime);
            values.put("status", "OPEN");
            values.put("created_at", System.currentTimeMillis());
            long roomId = database.insertOrThrow(
                    DatabaseHelper.TABLE_STUDY_ROOMS,
                    null,
                    values
            );
            insertDefaultSeats(database, roomId);
            database.setTransactionSuccessful();
            return roomId;
        } finally {
            database.endTransaction();
        }
    }
```

代码 11 自习室查询与新增（原文件共 172 行）

代码说明：查询通过 LEFT JOIN 汇总座位数和空闲数，并按楼栋、楼层排序。新增房间时同时生成默认座位。

论文写作提示：重点说明列表数据来自 SQLite，空座数是 SQL 聚合结果，不是写死文本。

自习室修改与删除

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\StudyRoomDao.java
范围	第 105 至 139 行
内容	updateRoom 与 deleteRoom

```
    public boolean updateRoom(
            long roomId,
            String name,
            String building,
            String floor,
            String location,
            String openTime,
            String closeTime
    ) {
        ContentValues values = new ContentValues();
        values.put("name", name.trim());
        values.put("building", building.trim());
        values.put("floor", floor.trim());
        values.put("location", location.trim());
        values.put("open_time", openTime);
        values.put("close_time", closeTime);
        return databaseHelper.getWritableDatabase().update(
                DatabaseHelper.TABLE_STUDY_ROOMS,
                values,
```

```
            "room_id = ?",
            new String[]{String.valueOf(roomId)}
    ) > 0;
}

public boolean deleteRoom(long roomId) {
    try {
        return databaseHelper.getWritableDatabase().delete(
                DatabaseHelper.TABLE_STUDY_ROOMS,
                "room_id = ?",
                new String[]{String.valueOf(roomId)}
        ) > 0;
    } catch (SQLiteConstraintException exception) {
        return false;
    }
}
```

代码 12 自习室修改与删除（原文件共 172 行）

代码说明：修改按主键更新房间资料。删除操作捕获外键约束异常，存在历史预约时拒绝破坏数据。

论文写作提示：体现 CRUD 的完整性以及外键保护策略。

ListView 自定义适配器

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\adapter\StudyRoomAdapter.java
范围	第 58 至 94 行
内容	StudyRoomAdapter.getView

```
public View getView(int position, View convertView, ViewGroup parent) {
    RoomViewHolder holder;
    if (convertView == null) {
        convertView = inflater.inflate(R.layout.item_study_room, parent, false);
        holder = new RoomViewHolder(convertView);
        convertView.setTag(holder);
    } else {
        holder = (RoomViewHolder) convertView.getTag();
    }

    StudyRoom room = getItem(position);
    holder.name.setText(room.getName());
    holder.location.setText(context.getString(
            R.string.room_location_format,
            room.getBuilding(),
            room.getFloor(),
            room.getLocation()
    ));
    holder.hours.setText(context.getString(
            R.string.room_hours_format,
            room.getOpenTime(),
            room.getCloseTime()
    ));
    holder.seats.setText(context.getString(
            R.string.room_seats_format,
            room.getAvailableSeats(),
            room.getTotalSeats()
    ));
    holder.status.setText(room.isOpen()
            ? R.string.room_status_open
            : R.string.room_status_closed);
    holder.status.setTextColor(ContextCompat.getColor(
            context,
```

```
                room.isOpen() ? R.color.app_primary : R.color.text_secondary
        ));
        return convertView;
    }
```

代码 13 ListView 自定义适配器（原文件共 111 行）

代码说明：把数据库查出的 StudyRoom 对象映射到列表卡片，使用 ViewHolder 保存控件引用，减少重复 findViewById。

论文写作提示：配合自习室列表截图，说明 ListView 与 Adapter 的配合。

第 4.4 节 智能选座

推荐算法采用可解释的多条件评分，不使用复杂机器学习模型，符合课程项目规模，也便于答辩说明。

多条件推荐评分

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\service\SeatRecommendationService.java
范围	第 18 至 79 行
内容	SeatRecommendationService.rank

```
    public List<RankedSeat> rank(
            List<Seat> seats,
            boolean preferQuiet,
            boolean preferWindow,
            boolean preferSocket
    ) {
        List<RankedSeat> available = new ArrayList<>();
        List<RankedSeat> unavailable = new ArrayList<>();

        for (Seat seat : seats) {
            if (!seat.isAvailable()) {
                String reason = seat.isPaused() ? "暂停使用" : "该时段已预约";
                unavailable.add(new RankedSeat(seat, 0, Collections.singletonList(reason)));
                continue;
            }

            int score = 50;
            List<String> reasons = new ArrayList<>();

            if (preferQuiet) {
                if (seat.getNoiseLevel() == 1) {
                    score += 35;
                    reasons.add("安静环境");
                } else if (seat.getNoiseLevel() == 2) {
                    score += 15;
                    reasons.add("较安静");
                }
                if (seat.getArea().contains("安静")) {
                    score += 15;
                    reasons.add("安静区");
                }
            }
            if (preferWindow && seat.isWindowSeat()) {
                score += 25;
                reasons.add("靠窗");
```

```
            }
            if (preferSocket && seat.hasSocket()) {
                score += 25;
                reasons.add("有插座");
            }
            if (reasons.isEmpty()) {
                reasons.add(seat.getArea());
            }
            available.add(new RankedSeat(seat, score, reasons));
        }

        available.sort((left, right) -> {
            int scoreOrder = Integer.compare(right.getScore(), left.getScore());
            if (scoreOrder != 0) {
                return scoreOrder;
            }
            return left.getSeat().getSeatNo().compareTo(right.getSeat().getSeatNo());
        });
        unavailable.sort((left, right) ->
                left.getSeat().getSeatNo().compareTo(right.getSeat().getSeatNo())
        );

        List<RankedSeat> result = new ArrayList<>(available.size() + unavailable.size());
        result.addAll(available);
        result.addAll(unavailable);
        return result;
    }
```

代码 14 多条件推荐评分（原文件共 80 行）

代码说明：只有满足预约时段且未暂停的座位会获得推荐分数。安静、靠窗和插座按用户偏好加权，生成可读推荐理由，并将不可用座位置底。

论文写作提示：建议配公式、偏好勾选前后排序变化截图和推荐理由截图。

座位实时占用查询

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\SeatDao.java
范围	第 25 至 61 行
内容	SeatDao.getSeats

```
    public List<Seat> getSeats(long roomId, long startAt, long endAt) {
        List<Seat> seats = new ArrayList<>();
        String sql = "SELECT s.seat_id, s.room_id, s.seat_no, s.area, " +
                "s.is_window, s.has_socket, s.noise_level, s.status, " +
                "CASE WHEN EXISTS (" +
                "SELECT 1 FROM " + DatabaseHelper.TABLE_RESERVATIONS + " r " +
                "WHERE r.seat_id = s.seat_id " +
                "AND r.status IN ('BOOKED', 'CHECKED_IN') " +
                "AND r.start_at < ? AND r.end_at > ?) " +
                "THEN 1 ELSE 0 END AS occupied " +
                "FROM " + DatabaseHelper.TABLE_SEATS + " s " +
                "WHERE s.room_id = ? ORDER BY s.seat_no";

        try (Cursor cursor = databaseHelper.getReadableDatabase().rawQuery(
                sql,
                new String[]{
                        String.valueOf(endAt),
                        String.valueOf(startAt),
                        String.valueOf(roomId)
                }
```

```
        )) {
            while (cursor.moveToNext()) {
                seats.add(new Seat(
                        cursor.getLong(0),
                        cursor.getLong(1),
                        cursor.getString(2),
                        cursor.getString(3),
                        cursor.getInt(4) == 1,
                        cursor.getInt(5) == 1,
                        cursor.getInt(6),
                        cursor.getString(7),
                        cursor.getInt(8) == 1
                ));
            }
        }
        return seats;
    }
```

代码 15 座位实时占用查询（原文件共 165 行）

代码说明：通过相关子查询检查 BOOKED 和 CHECKED_IN 预约是否与目标时段重叠，界面展示的是实时数据库状态，而不是静态图片。

论文写作提示：与预约冲突规则配合说明左闭右开区间。

24 小时制自定义时间

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\activity\SeatSelectionActivity.java
范围	第 241 至 333 行
内容	初始化时间、时间选择、时间段计算

```
    private void initializeDefaultTimeRange() {
        Calendar now = Calendar.getInstance();
        boolean isToday = now.get(Calendar.YEAR) == selectedDate.get(Calendar.YEAR)
                && now.get(Calendar.DAY_OF_YEAR) == selectedDate.get(Calendar.DAY_OF_YEAR);
        if (isToday) {
            int currentMinute = now.get(Calendar.HOUR_OF_DAY) * 60
                    + now.get(Calendar.MINUTE);
            int roundedMinute = ((currentMinute + 14) / 15) * 15;
            if (roundedMinute >= 24 * 60) {
                selectedDate.add(Calendar.DAY_OF_MONTH, 1);
                resetTime(selectedDate);
                selectedStartMinute = 0;
            } else {
                selectedStartMinute = roundedMinute;
            }
        } else {
            selectedStartMinute = 8 * 60;
        }
        selectedEndMinute = Math.min(23 * 60 + 59, selectedStartMinute + 120);
        if (selectedEndMinute <= selectedStartMinute) {
            selectedStartMinute = Math.max(0, selectedEndMinute - 30);
        }
    }

    private void showStartTimePicker() {
        new TimePickerDialog(
                this,
                (view, hourOfDay, minute) -> {
                    int value = hourOfDay * 60 + minute;
                    if (value >= selectedEndMinute) {
```

```
                    Toast.makeText(this, R.string.error_time_range, Toast.LENGTH_SHORT).show();
                    return;
                }
                selectedStartMinute = value;
                updateTimeButtons();
                loadSeats();
            },
            selectedStartMinute / 60,
            selectedStartMinute % 60,
            true
    ).show();
}

private void showEndTimePicker() {
    new TimePickerDialog(
            this,
            (view, hourOfDay, minute) -> {
                int value = hourOfDay * 60 + minute;
                if (value <= selectedStartMinute) {
                    Toast.makeText(this, R.string.error_time_range, Toast.LENGTH_SHORT).show();
                    return;
                }
                selectedEndMinute = value;
                updateTimeButtons();
                loadSeats();
            },
            selectedEndMinute / 60,
            selectedEndMinute % 60,
            true
    ).show();
}

private void updateTimeButtons() {
    startTimeButton.setText(getString(
            R.string.time_button_start_format,
            formatMinuteOfDay(selectedStartMinute)
    ));
    endTimeButton.setText(getString(
            R.string.time_button_end_format,
            formatMinuteOfDay(selectedEndMinute)
    ));
}

private void updateSelectedTimeRange() {
    resetTime(selectedDate);
    selectedStartAt = selectedDate.getTimeInMillis()
            + selectedStartMinute * 60_000L;
    selectedEndAt = selectedDate.getTimeInMillis()
            + selectedEndMinute * 60_000L;
}

private void resetTime(Calendar calendar) {
    calendar.set(Calendar.HOUR_OF_DAY, 0);
    calendar.set(Calendar.MINUTE, 0);
    calendar.set(Calendar.SECOND, 0);
    calendar.set(Calendar.MILLISECOND, 0);
}

private String formatSelectedTimeRange() {
    return formatMinuteOfDay(selectedStartMinute)
            + "–"
            + formatMinuteOfDay(selectedEndMinute);
}
```

代码 16 24 小时制自定义时间（原文件共 345 行）

代码说明：默认时间取当前时间之后的 15 分钟起点，开始和结束时间分别通过 24 小时制 TimePicker 选择，最终换算为毫秒级时间区间。

论文写作提示：用界面截图说明可以精确选择到分钟，并说明结束时间必须晚于开始时间。

第 4.5 节 预约与冲突控制

预约不是简单 INSERT，而是在数据库事务中重新检查开放时间、座位冲突和用户冲突。

预约时间规则

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\service\ReservationRules.java
范围	第 13 至 48 行
内容	intervalsOverlap 与 isWithinOpenHours

```
public static boolean intervalsOverlap(
        long firstStart,
        long firstEnd,
        long secondStart,
        long secondEnd
) {
    return firstStart < secondEnd && secondStart < firstEnd;
}

public static boolean isWithinOpenHours(
        long startAt,
        long endAt,
        String openTime,
        String closeTime
) {
    int openMinute = parseMinutes(openTime);
    int closeMinute = parseMinutes(closeTime);
    if (openMinute < 0 || closeMinute < 0 || openMinute >= closeMinute) {
        return false;
    }

    Calendar start = Calendar.getInstance();
    start.setTimeInMillis(startAt);
    Calendar end = Calendar.getInstance();
    end.setTimeInMillis(endAt);
    if (start.get(Calendar.YEAR) != end.get(Calendar.YEAR)
            || start.get(Calendar.DAY_OF_YEAR) != end.get(Calendar.DAY_OF_YEAR)) {
        return false;
    }

    int startMinute = start.get(Calendar.HOUR_OF_DAY) * 60
            + start.get(Calendar.MINUTE);
    int endMinute = end.get(Calendar.HOUR_OF_DAY) * 60
            + end.get(Calendar.MINUTE);
    return startMinute >= openMinute && endMinute <= closeMinute;
}
```

代码 17 预约时间规则（原文件共 69 行）

代码说明：冲突公式使用左闭右开区间，首尾恰好相接不算冲突；开放时间校验同时比较日期、开始分钟和结束分钟。

论文写作提示：建议把公式与边界测试用例一起放进预约章节。

预约事务与写入

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\ReservationDao.java
范围	第 72 至 132 行
内容	createReservation

```
    public BookingResult createReservation(
            long userId,
            long seatId,
            long startAt,
            long endAt
    ) {
        if (startAt >= endAt) {
            return BookingResult.failure(BookingFailure.INVALID_TIME_RANGE);
        }
        if (startAt <= System.currentTimeMillis()) {
            return BookingResult.failure(BookingFailure.STARTED_IN_PAST);
        }

        SQLiteDatabase database = databaseHelper.getWritableDatabase();
        database.beginTransaction();
        try {
            SeatContext context = loadSeatContext(database, seatId);
            if (context == null) {
                return BookingResult.failure(BookingFailure.SEAT_NOT_FOUND);
            }
            if (!"OPEN".equals(context.roomStatus)) {
                return BookingResult.failure(BookingFailure.ROOM_CLOSED);
            }
            if (!"AVAILABLE".equals(context.seatStatus)) {
                return BookingResult.failure(BookingFailure.SEAT_PAUSED);
            }
            if (!ReservationRules.isWithinOpenHours(
                    startAt,
                    endAt,
                    context.openTime,
                    context.closeTime
            )) {
                return BookingResult.failure(BookingFailure.OUTSIDE_OPEN_HOURS);
            }
            if (hasConflict(database, "seat_id", seatId, startAt, endAt)) {
                return BookingResult.failure(BookingFailure.SEAT_CONFLICT);
            }
            if (hasConflict(database, "user_id", userId, startAt, endAt)) {
                return BookingResult.failure(BookingFailure.USER_CONFLICT);
            }

            ContentValues values = new ContentValues();
            values.put("user_id", userId);
            values.put("seat_id", seatId);
            values.put("start_at", startAt);
            values.put("end_at", endAt);
            values.put("status", "BOOKED");
            values.put("created_at", System.currentTimeMillis());
            long reservationId = database.insertOrThrow(
                    DatabaseHelper.TABLE_RESERVATIONS,
                    null,
                    values
            );
            database.setTransactionSuccessful();
            return BookingResult.success(reservationId);
        } catch (SQLiteException exception) {
            return BookingResult.failure(BookingFailure.DATABASE_ERROR);
        } finally {
```

```
            database.endTransaction();
        }
    }
```

代码 18 预约事务与写入（原文件共 281 行）

代码说明：在同一个 SQLite 事务中完成座位状态、开放时间、座位冲突和用户冲突检查，再插入预约，避免两次点击或多个检查之间出现竞态。

论文写作提示：重点说明“界面校验不等于最终校验”，真正的约束仍在数据层。

座位与用户冲突查询

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\ReservationDao.java
范围	第 197 至 217 行
内容	hasConflict

```
    private boolean hasConflict(
            SQLiteDatabase database,
            String column,
            long value,
            long startAt,
            long endAt
    ) {
        try (Cursor cursor = database.rawQuery(
                "SELECT COUNT(*) FROM " + DatabaseHelper.TABLE_RESERVATIONS +
                        " WHERE " + column + " = ? " +
                        "AND status IN ('BOOKED', 'CHECKED_IN') " +
                        "AND start_at < ? AND end_at > ?",
                new String[]{
                        String.valueOf(value),
                        String.valueOf(endAt),
                        String.valueOf(startAt)
                }
        )) {
            return cursor.moveToFirst() && cursor.getInt(0) > 0;
        }
    }
```

代码 19 座位与用户冲突查询（原文件共 281 行）

代码说明：同一查询方法可以分别按 seat_id 或 user_id 检查有效预约。取消和爽约记录不会继续占用时间。

论文写作提示：可以说明数据库状态值对冲突判定结果有直接影响。

第 4.6 节 签到、签退与爽约

签到模块用时间窗口控制状态流转，并修复了未来预约被误判为爽约的问题。

签到时间窗口

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\service\CheckInRules.java
范围	第 20 至 40 行
内容	签到窗口、时长、爽约截止规则

```
public static Window evaluateWindow(long now, long reservationStart) {
    if (now < reservationStart - OPEN_BEFORE_MS) {
        return Window.TOO_EARLY;
    }
    if (now > reservationStart + LATE_AFTER_MS) {
        return Window.TOO_LATE;
    }
    return Window.OPEN;
}

public static int calculateActualMinutes(long checkinAt, long checkoutAt) {
    return (int) Math.max(1L, (checkoutAt - checkinAt + 59_999L) / 60_000L);
}

public static long noShowCutoff(long now) {
    return now - LATE_AFTER_MS;
}

public static boolean isExpiredNoShow(long reservationStart, long now) {
    return reservationStart < noShowCutoff(now);
}
```

代码 20 签到时间窗口（原文件共 41 行）

代码说明：预约开始前 15 分钟开放签到，开始后 20 分钟仍未签到则过期；实际学习时长按分钟向上取整。

论文写作提示：结合示意图说明 BOOKED、CHECKED_IN、COMPLETED 和 NO_SHOW 的转换。

签到事务

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\CheckInDao.java
范围	第 67 至 102 行
内容	CheckInDao.checkIn

```
public OperationResult checkIn(long userId, long reservationId, long now) {
    SQLiteDatabase database = databaseHelper.getWritableDatabase();
    database.beginTransaction();
    try {
        ReservationStatus status = loadStatus(database, userId, reservationId);
        if (status == null) {
            return OperationResult.failure(Failure.RESERVATION_NOT_FOUND);
        }
        if (!"BOOKED".equals(status.status)) {
            return OperationResult.failure(Failure.INVALID_STATUS);
        }
```

```
            CheckInRules.Window window = CheckInRules.evaluateWindow(now, status.startAt);
            if (window == CheckInRules.Window.TOO_EARLY) {
                return OperationResult.failure(Failure.TOO_EARLY);
            }
            if (window == CheckInRules.Window.TOO_LATE) {
                updateStatus(database, reservationId, "NO_SHOW");
                database.setTransactionSuccessful();
                return OperationResult.failure(Failure.TOO_LATE);
            }

            updateStatus(database, reservationId, "CHECKED_IN");
            ContentValues record = new ContentValues();
            record.put("reservation_id", reservationId);
            record.put("checkin_at", now);
            record.put("actual_minutes", 0);
            record.put("created_at", now);
            database.insertOrThrow(DatabaseHelper.TABLE_CHECKIN_RECORDS, null, record);
            database.setTransactionSuccessful();
            return OperationResult.success(0);
        } catch (SQLiteException exception) {
            return OperationResult.failure(Failure.DATABASE_ERROR);
        } finally {
            database.endTransaction();
        }
    }
```

代码 21 签到事务（原文件共 313 行）

代码说明：在事务中检查预约状态和签到时间，状态改为 CHECKED_IN 后插入唯一签到记录，避免重复签到。

论文写作提示：建议展示提前签到、正常签到和重复签到三种测试结果。

签退与学习时长

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\CheckInDao.java
范围	第 104 至 140 行
内容	CheckInDao.checkOut

```
    public OperationResult checkOut(long userId, long reservationId, long now) {
        SQLiteDatabase database = databaseHelper.getWritableDatabase();
        database.beginTransaction();
        try {
            CheckoutContext context = loadCheckoutContext(database, userId, reservationId);
            if (context == null) {
                return OperationResult.failure(Failure.RESERVATION_NOT_FOUND);
            }
            if (!"CHECKED_IN".equals(context.status) || context.checkinAt <= 0) {
                return OperationResult.failure(Failure.NOT_CHECKED_IN);
            }

            int actualMinutes = CheckInRules.calculateActualMinutes(
                    context.checkinAt,
                    now
            );
            ContentValues recordValues = new ContentValues();
            recordValues.put("checkout_at", now);
            recordValues.put("actual_minutes", actualMinutes);
            int recordChanged = database.update(
                    DatabaseHelper.TABLE_CHECKIN_RECORDS,
                    recordValues,
                    "reservation_id = ?",
                    new String[]{String.valueOf(reservationId)}
```

```
            );
            if (recordChanged != 1) {
                return OperationResult.failure(Failure.DATABASE_ERROR);
            }
            updateStatus(database, reservationId, "COMPLETED");
            database.setTransactionSuccessful();
            return OperationResult.success(actualMinutes);
        } catch (SQLiteException exception) {
            return OperationResult.failure(Failure.DATABASE_ERROR);
        } finally {
            database.endTransaction();
        }
    }
```

代码 22 签退与学习时长（原文件共 313 行）

代码说明：签退时写入 checkout_at 和实际学习分钟数，预约状态变为 COMPLETED。

论文写作提示：与签到记录表字段说明一起展示。

爽约修复与状态维护

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\CheckInDao.java
范围	第 142 至 198 行
内容	误标修复、过期爽约、管理端刷新

```
    public int repairPrematureNoShows(long userId, long now) {
        long cutoff = CheckInRules.noShowCutoff(now);
        try (SQLiteStatement statement = databaseHelper.getWritableDatabase()
                .compileStatement(
                        "UPDATE " + DatabaseHelper.TABLE_RESERVATIONS +
                                " SET status = 'BOOKED' " +
                                "WHERE user_id = ? AND status = 'NO_SHOW' " +
                                "AND start_at >= ?"
                )) {
            statement.bindLong(1, userId);
            statement.bindLong(2, cutoff);
            return statement.executeUpdateDelete();
        }
    }

    public int repairAllPrematureNoShows(long now) {
        long cutoff = CheckInRules.noShowCutoff(now);
        try (SQLiteStatement statement = databaseHelper.getWritableDatabase()
                .compileStatement(
                        "UPDATE " + DatabaseHelper.TABLE_RESERVATIONS +
                                " SET status = 'BOOKED' " +
                                "WHERE status = 'NO_SHOW' AND start_at >= ?"
                )) {
            statement.bindLong(1, cutoff);
            return statement.executeUpdateDelete();
        }
    }

    public int markExpiredNoShows(long userId, long now) {
        // Bind Long values: SQLite has no INTEGER affinity on start_at - grace expressions.
        long cutoff = CheckInRules.noShowCutoff(now);
        try (SQLiteStatement statement = databaseHelper.getWritableDatabase()
                .compileStatement(
                        "UPDATE " + DatabaseHelper.TABLE_RESERVATIONS +
                                " SET status = 'NO_SHOW' " +
                                "WHERE user_id = ? AND status = 'BOOKED' " +
```

```
                                "AND start_at < ?"
                )) {
            statement.bindLong(1, userId);
            statement.bindLong(2, cutoff);
            return statement.executeUpdateDelete();
        }
    }

    public int markAllExpiredNoShows(long now) {
        // Use a precomputed numeric cutoff so future reservations remain BOOKED.
        long cutoff = CheckInRules.noShowCutoff(now);
        try (SQLiteStatement statement = databaseHelper.getWritableDatabase()
                .compileStatement(
                        "UPDATE " + DatabaseHelper.TABLE_RESERVATIONS +
                                " SET status = 'NO_SHOW' " +
                                "WHERE status = 'BOOKED' AND start_at < ?"
                )) {
            statement.bindLong(1, cutoff);
            return statement.executeUpdateDelete();
        }
    }
```

代码 23 爽约修复与状态维护（原文件共 313 行）

代码说明：先恢复尚未到截止时间却被误标的 NO_SHOW 记录，再把真正超过截止时间的 BOOKED 记录更新为 NO_SHOW。绑定数值参数避免 SQLite 类型误判。

论文写作提示：这是调试章节很好的证据，可以说明如何定位 SQL 参数类型问题并增加回归测试。

签到页面自动刷新

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\activity\CheckInActivity.java
范围	第 79 至 92 行
内容	onResume 与 onPause 的定时刷新

```
    protected void onResume() {
        super.onResume();
        if (reservationDao != null) {
            loadReservation();
            refreshHandler.removeCallbacks(refreshTask);
            refreshHandler.postDelayed(refreshTask, 15_000L);
        }
    }

    @Override
    protected void onPause() {
        refreshHandler.removeCallbacks(refreshTask);
        super.onPause();
    }
```

代码 24 签到页面自动刷新（原文件共 323 行）

代码说明：页面停留在签到界面时每 15 秒重新检查一次时间，因此即使进入页面时还没到开放时间，到点后按钮也会自动变为可签到。

论文写作提示：说明页面时间状态不能只在首次打开时计算一次。

第 4.7 节 专注计时与每日目标

专注计时使用时间戳恢复，而不是只依赖内存中的递减变量。

时间戳恢复

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\model\FocusTimerState.java
范围	第 82 至 87 行
内容	FocusTimerState.getElapsedMs

```
public long getElapsedMs(long now) {
    long runningPart = isRunning()
            ? Math.max(0L, now - runStartedAt)
            : 0L;
    return Math.min(getTargetDurationMs(), elapsedBeforeRunMs + runningPart);
}
```

代码 25 时间戳恢复（原文件共 88 行）

代码说明：运行状态用“本次运行开始时间”计算已经过时间，暂停状态只使用已保存的累计时间。结果封顶到目标时长。

论文写作提示：重点说明切换页面、锁屏和 Activity 重建时不会依赖静态倒计时变量。

计时状态持久化

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\util\FocusStateStore.java
范围	第 35 至 65 行
内容	FocusStateStore.save 与 load

```
public void save(FocusTimerState state) {
    preferences.edit()
            .putBoolean(KEY_ACTIVE, true)
            .putBoolean(KEY_RUNNING, state.isRunning())
            .putInt(KEY_DURATION, state.getDurationMinutes())
            .putLong(KEY_ELAPSED_BEFORE, state.getElapsedBeforeRunMs())
            .putLong(KEY_RUN_STARTED, state.getRunStartedAt())
            .putLong(KEY_SESSION_STARTED, state.getSessionStartedAt())
            .putString(KEY_SUBJECT, state.getSubject())
            .putString(KEY_TASK, state.getTask())
            .putString(KEY_TOKEN, state.getSessionToken())
            .commit();
}

public FocusTimerState load() {
    if (!preferences.getBoolean(KEY_ACTIVE, false)) {
```

```
            return null;
        }
        return new FocusTimerState(
                preferences.getBoolean(KEY_RUNNING, false)
                        ? FocusTimerState.Status.RUNNING
                        : FocusTimerState.Status.PAUSED,
                preferences.getInt(KEY_DURATION, 25),
                preferences.getLong(KEY_ELAPSED_BEFORE, 0),
                preferences.getLong(KEY_RUN_STARTED, 0),
                preferences.getLong(KEY_SESSION_STARTED, 0),
                preferences.getString(KEY_SUBJECT, ""),
                preferences.getString(KEY_TASK, ""),
                preferences.getString(KEY_TOKEN, "")
        );
    }
```

代码 26 计时状态持久化（原文件共 70 行）

代码说明：保存状态、时长、已累计时间、启动时间、科目、任务和唯一 token；恢复时据此重新计算剩余时间。

论文写作提示：可与 Activity 生命周期和 SharedPreferences 的用途一起说明。

专注结束与唯一写入

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\activity\FocusTimerActivity.java
范围	第 277 至 332 行
内容	FocusTimerActivity.finishSession

```
    private void finishSession(boolean completedAutomatically) {
        if (state == null) {
            return;
        }
        tickerHandler.removeCallbacks(ticker);
        FocusTimerState finishedState = state;
        long now = System.currentTimeMillis();
        long elapsed = finishedState.getElapsedMs(now);
        int minutes = Math.max(1, (int) ((elapsed + 59_999L) / 60_000L));
        StudySessionDao.SaveResult saveResult = sessionDao.saveFocusSession(
                userId,
                finishedState.getSubject(),
                finishedState.getTask(),
                finishedState.getSessionStartedAt(),
                now,
                minutes,
                finishedState.getSessionToken()
        );

        if (saveResult == StudySessionDao.SaveResult.FAILED) {
            state = new FocusTimerState(
                    FocusTimerState.Status.PAUSED,
                    finishedState.getDurationMinutes(),
                    Math.min(elapsed, finishedState.getTargetDurationMs()),
                    0,
                    finishedState.getSessionStartedAt(),
                    finishedState.getSubject(),
                    finishedState.getTask(),
                    finishedState.getSessionToken()
            );
            stateStore.save(state);
            renderState(now);
            new MaterialAlertDialogBuilder(this)
                    .setTitle(R.string.focus_save_failed_title)
```

```
                .setMessage(getString(R.string.focus_save_failed_message, minutes))
                .setPositiveButton(R.string.action_retry, null)
                .show();
        return;
    }

    stateStore.clear();
    state = null;
    renderIdle();
    updateDailyGoal();

    int message = saveResult == StudySessionDao.SaveResult.SAVED
            ? R.string.focus_saved_message
            : R.string.focus_already_saved_message;
    new MaterialAlertDialogBuilder(this)
            .setTitle(completedAutomatically
                    ? R.string.focus_complete_title
                    : R.string.focus_saved_title)
            .setMessage(getString(message, minutes))
            .setPositiveButton(R.string.action_done, null)
            .show();
}
```

代码 27 专注结束与唯一写入（原文件共 458 行）

代码说明：结束专注时计算实际分钟，用 sessionToken 写入学习记录。即使因异常重复执行保存，同一会话也只会成功写入一次。

论文写作提示：论文中可强调业务防重复设计与数据库唯一索引的配合。

第 5 章 RecyclerView 自学控件

RecyclerView 用于“我的预约”列表，是课程要求中单独讲解的自学控件。

RecyclerView Adapter 与 ViewHolder

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\adapter\ReservationAdapter.java
范围	第 52 至 102 行
内容	刷新、创建 ViewHolder、绑定数据

```
public void setReservations(List<Reservation> newReservations) {
    int oldSize = reservations.size();
    reservations.clear();
    if (oldSize > 0) {
        notifyItemRangeRemoved(0, oldSize);
    }
    reservations.addAll(newReservations);
    if (!reservations.isEmpty()) {
        notifyItemRangeInserted(0, reservations.size());
    }
}

@NonNull
@Override
public ReservationViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_reservation, parent, false);
```

```
        return new ReservationViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull ReservationViewHolder holder, int position) {
        Reservation reservation = reservations.get(position);
        holder.room.setText(reservation.getRoomName());
        holder.seat.setText(context.getString(
                R.string.reservation_seat_format,
                reservation.getSeatNo(),
                reservation.getSeatArea()
        ));
        holder.schedule.setText(formatSchedule(reservation.getStartAt(), reservation.getEndAt()));
        holder.number.setText(context.getString(
                R.string.reservation_number_format,
                reservation.getReservationId()
        ));
        bindStatus(holder, reservation.getStatus());

        boolean cancellable = reservation.canCancel(System.currentTimeMillis());
        holder.cancel.setVisibility(cancellable ? View.VISIBLE : View.GONE);
        holder.cancel.setOnClickListener(view -> cancelListener.onCancelRequested(reservation));

        if ("BOOKED".equals(reservation.getStatus())) {
            holder.action.setText(R.string.action_open_check_in);
            holder.action.setVisibility(View.VISIBLE);
        } else if ("CHECKED_IN".equals(reservation.getStatus())) {
            holder.action.setText(R.string.action_check_out);
            holder.action.setVisibility(View.VISIBLE);
        } else {
            holder.action.setVisibility(View.GONE);
        }
        holder.action.setOnClickListener(view -> actionListener.onActionRequested(reservation));
    }
```

代码 28 RecyclerView Adapter 与 ViewHolder（原文件共 169 行）

代码说明：Adapter 管理预约数据列表，ViewHolder 保存卡片中的 TextView 和 Button 引用。绑定数据时还根据预约状态决定签到、取消按钮是否显示。

论文写作提示：建议配 activity_my_booking.xml、item_reservation.xml 和运行截图，并在报告第 5 章介绍 Adapter、ViewHolder、LayoutManager、刷新和点击事件。

第 4.8 节 学习统计

统计页面不生成随机数据，而是对 study_sessions、reservations、checkin_records 和 study_goals 做聚合。

每日与到座率统计

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\StudyStatisticsDao.java
范围	第 71 至 119 行

内容	getDailyMinutes 与 getAttendanceStats

```
public Map<String, Integer> getDailyMinutes(long userId, long startAt, long endAt) {
    Map<String, Integer> result = new HashMap<>();
    try (Cursor cursor = databaseHelper.getReadableDatabase().rawQuery(
            "SELECT start_at, duration_minutes FROM " +
                    DatabaseHelper.TABLE_STUDY_SESSIONS +
                    " WHERE user_id = ? AND start_at >= ? AND start_at < ?",
            new String[]{
                    String.valueOf(userId),
                    String.valueOf(startAt),
                    String.valueOf(endAt)
            }
    )) {
        while (cursor.moveToNext()) {
            String key = TimeUtil.dateKey(cursor.getLong(0));
            result.put(key, result.getOrDefault(key, 0) + cursor.getInt(1));
        }
    }
    return result;
}

public AttendanceStats getAttendanceStats(long userId, long startAt, long endAt) {
    int attended = 0;
    int expected = 0;
    int noShow = 0;
    try (Cursor cursor = databaseHelper.getReadableDatabase().rawQuery(
            "SELECT status FROM " + DatabaseHelper.TABLE_RESERVATIONS +
                    " WHERE user_id = ? AND end_at >= ? AND end_at < ?",
            new String[]{
                    String.valueOf(userId),
                    String.valueOf(startAt),
                    String.valueOf(endAt)
            }
    )) {
        while (cursor.moveToNext()) {
            String status = cursor.getString(0);
            if ("CANCELLED".equals(status)) {
                continue;
            }
            expected++;
            if ("COMPLETED".equals(status) || "CHECKED_IN".equals(status)) {
                attended++;
            }
            if ("NO_SHOW".equals(status)) {
                noShow++;
            }
        }
    }
    return new AttendanceStats(attended, expected, noShow);
}
```

代码 29 每日与到座率统计（原文件共 197 行）

代码说明：每日学习时长按 start_at 日期分组；到座率统计排除 CANCELLED，只把已完成或使用中的预约计为到座。

论文写作提示：说明统计分母和特殊情况处理，避免只展示一个不明来源的百分比。

连续学习天数

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\StudyStatisticsDao.java

范围	第 186 至 196 行
内容	getStreakDays

```
    public int getStreakDays(long userId, long now) {
        java.util.Calendar rangeStart = java.util.Calendar.getInstance();
        rangeStart.setTimeInMillis(now);
        rangeStart.add(java.util.Calendar.DAY_OF_MONTH, -365);
        Map<String, Integer> days = getDailyMinutes(
                userId,
                TimeUtil.startOfDay(rangeStart.getTimeInMillis()),
                TimeUtil.tomorrowStartMillis()
        );
        return StatisticsRules.calculateStreak(days, TimeUtil.todayKey());
    }
```

代码 30 连续学习天数（原文件共 197 行）

代码说明：从今天开始向前检查每日学习记录；如果今天还没有记录，则从昨天开始计算，避免上午打开统计时错误显示连续天数归零。

论文写作提示：可配首页连续天数与统计页数据一致的截图。

个性化学习建议

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\service\StatisticsRules.java
范围	第 47 至 69 行
内容	StatisticsRules.buildSuggestion

```
    public static StudyStatisticsSnapshot.Suggestion buildSuggestion(
            int monthMinutes,
            int noShowRate,
            int goalRate,
            StudyStatisticsSnapshot.ActivePeriod activePeriod
    ) {
        if (monthMinutes == 0) {
            return StudyStatisticsSnapshot.Suggestion.NO_DATA;
        }
        if (noShowRate >= 25) {
            return StudyStatisticsSnapshot.Suggestion.HIGH_NO_SHOW;
        }
        if (goalRate < 50) {
            return StudyStatisticsSnapshot.Suggestion.LOW_GOAL_COMPLETION;
        }
        if (activePeriod == StudyStatisticsSnapshot.ActivePeriod.EVENING) {
            return StudyStatisticsSnapshot.Suggestion.EVENING_FOCUS;
        }
        if (activePeriod == StudyStatisticsSnapshot.ActivePeriod.MORNING) {
            return StudyStatisticsSnapshot.Suggestion.MORNING_FOCUS;
        }
        return StudyStatisticsSnapshot.Suggestion.KEEP_RHYTHM;
    }
```

代码 31 个性化学习建议（原文件共 70 行）

代码说明：建议有明确优先级：先检查是否有学习数据，再检查爽约率、目标完成率和常用时段。所有建议都由真实统计值触发。

论文写作提示：说明本项目中的“智能”是可解释规则，不是虚构的大模型。

自绘学习柱状图

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\view\StudyBarChartView.java
范围	第 69 至 132 行
内容	StudyBarChartView.onDraw

```
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        if (points.isEmpty()) {
            return;
        }

        float left = dp(10);
        float right = getWidth() - dp(10);
        float top = dp(30);
        float bottom = getHeight() - dp(34);
        float chartHeight = bottom - top;
        float slotWidth = (right - left) / points.size();
        float barWidth = slotWidth * 0.48f;
        int maxMinutes = 30;
        int total = 0;
        for (DailyStudyPoint point : points) {
            maxMinutes = Math.max(maxMinutes, point.getMinutes());
            total += point.getMinutes();
        }

        for (int line = 0; line <= 3; line++) {
            float y = top + chartHeight * line / 3f;
            canvas.drawLine(left, y, right, y, gridPaint);
        }

        textPaint.setTextSize(sp(10));
        for (int index = 0; index < points.size(); index++) {
            DailyStudyPoint point = points.get(index);
            float centerX = left + slotWidth * index + slotWidth / 2f;
            float barHeight = chartHeight * point.getMinutes() / maxMinutes;
            float barTop = bottom - barHeight;

            if (point.getMinutes() > 0) {
                barRect.set(
                        centerX - barWidth / 2f,
                        barTop,
                        centerX + barWidth / 2f,
                        bottom
                );
                canvas.drawRoundRect(barRect, dp(5), dp(5), barPaint);
                canvas.drawText(
                        String.valueOf(point.getMinutes()),
                        centerX,
                        Math.max(top + sp(11), barTop - dp(5)),
                        textPaint
                );
            } else {
                canvas.drawLine(centerX - barWidth / 2f, bottom, centerX + barWidth / 2f,
                        bottom, gridPaint);
            }
```

```
            canvas.drawText(point.getLabel(), centerX, bottom + dp(20), textPaint);
        }

        if (total == 0) {
            textPaint.setColor(ContextCompat.getColor(getContext(), R.color.text_secondary));
            textPaint.setTextSize(sp(12));
            canvas.drawText(
                    getContext().getString(R.string.chart_empty),
                    getWidth() / 2f,
                    top + chartHeight / 2f,
                    textPaint
            );
        }
    }
```

代码 32 自绘学习柱状图（原文件共 141 行）

代码说明：通过 Canvas 绘制坐标网格、柱形、分钟数和日期标签，不需要第三方图表库。

论文写作提示：可与自定义 View 的测量和绘制流程一起说明。

第 4.9 节 主题与字号

所有页面继承 BaseActivity，在创建前应用统一主题和字体缩放。

全局主题与字号管理

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\util\ThemeManager.java
范围	第 19 至 67 行
内容	应用主题、切换主题、字号

```
    public static Context wrapFontScale(Context base) {
        Configuration configuration = new Configuration(
                base.getResources().getConfiguration()
        );
        configuration.fontScale = AppearancePreferences.getFontScale(base);
        return base.createConfigurationContext(configuration);
    }

    public static void applyActivityTheme(Activity activity) {
        if (AppearancePreferences.getThemeMode(activity)
                == AppearancePreferences.MODE_EYE_GREEN) {
            activity.setTheme(R.style.Theme_ZhiXiPro_EyeCare);
        } else {
            activity.setTheme(R.style.Theme_ZhiXiPro);
        }
    }

    public static void applySavedModeToApplication(Context context) {
        int mode = AppearancePreferences.getThemeMode(context);
        AppCompatDelegate.setDefaultNightMode(
                mode == AppearancePreferences.MODE_DARK
                        ? AppCompatDelegate.MODE_NIGHT_YES
                        : AppCompatDelegate.MODE_NIGHT_NO
        );
    }

    public static void changeTheme(Activity activity, int newMode) {
```

```
        int oldMode = AppearancePreferences.getThemeMode(activity);
        if (oldMode == newMode) {
            return;
        }
        AppearancePreferences.setThemeMode(activity, newMode);
        int targetNightMode = newMode == AppearancePreferences.MODE_DARK
                ? AppCompatDelegate.MODE_NIGHT_YES
                : AppCompatDelegate.MODE_NIGHT_NO;
        boolean dayNightChanged = AppCompatDelegate.getDefaultNightMode() != targetNightMode;
        AppCompatDelegate.setDefaultNightMode(targetNightMode);
        if (!dayNightChanged) {
            activity.recreate();
        }
    }

    public static void changeFontScale(Activity activity, float scale) {
        if (Float.compare(AppearancePreferences.getFontScale(activity), scale) == 0) {
            return;
        }
        AppearancePreferences.setFontScale(activity, scale);
        activity.recreate();
    }
```

代码 33 全局主题与字号管理（原文件共 68 行）

代码说明：ThemeManager 根据 SharedPreferences 选择日间、夜间或护眼绿主题，并在字号改变后重建当前 Activity，使页面立即使用新字号。

论文写作提示：可配三种主题截图和四种字号对比图。

设置页交互

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\activity\SettingsActivity.java
范围	第 50 至 109 行
内容	SettingsActivity.bindViews

```
    private void bindViews() {
        findViewById(R.id.button_settings_back).setOnClickListener(view -> finish());
        findViewById(R.id.button_settings_profile).setOnClickListener(view -> {
            Intent intent = new Intent(this, ProfileActivity.class);
            intent.putExtra(LoginActivity.EXTRA_USER_ID, userId);
            startActivity(intent);
        });
        findViewById(R.id.button_settings_admin).setOnClickListener(view -> {
            Intent intent = new Intent(this, AdminActivity.class);
            intent.putExtra(LoginActivity.EXTRA_USER_ID, userId);
            startActivity(intent);
        });
        findViewById(R.id.button_settings_about).setOnClickListener(view ->
                startActivity(new Intent(this, AboutActivity.class))
        );

        RadioGroup themeGroup = findViewById(R.id.group_theme_mode);
        int mode = AppearancePreferences.getThemeMode(this);
        if (mode == AppearancePreferences.MODE_DARK) {
            themeGroup.check(R.id.radio_theme_dark);
        } else if (mode == AppearancePreferences.MODE_EYE_GREEN) {
            themeGroup.check(R.id.radio_theme_eye);
        } else {
            themeGroup.check(R.id.radio_theme_light);
        }
```

```
        themeGroup.setOnCheckedChangeListener((group, checkedId) -> {
            if (checkedId == R.id.radio_theme_dark) {
                ThemeManager.changeTheme(this, AppearancePreferences.MODE_DARK);
            } else if (checkedId == R.id.radio_theme_eye) {
                ThemeManager.changeTheme(this, AppearancePreferences.MODE_EYE_GREEN);
            } else {
                ThemeManager.changeTheme(this, AppearancePreferences.MODE_LIGHT);
            }
        });

        MaterialButtonToggleGroup fontGroup = findViewById(R.id.group_font_size);
        fontGroup.check(fontButtonForScale(AppearancePreferences.getFontScale(this)));
        fontGroup.addOnButtonCheckedListener((group, checkedId, isChecked) -> {
            if (!isChecked) {
                return;
            }
            float scale;
            if (checkedId == R.id.button_font_small) {
                scale = AppearancePreferences.FONT_SMALL;
            } else if (checkedId == R.id.button_font_large) {
                scale = AppearancePreferences.FONT_LARGE;
            } else if (checkedId == R.id.button_font_xlarge) {
                scale = AppearancePreferences.FONT_EXTRA_LARGE;
            } else {
                scale = AppearancePreferences.FONT_NORMAL;
            }
            ThemeManager.changeFontScale(this, scale);
        });

        MaterialSwitch reminderSwitch = findViewById(R.id.switch_reminder);
        reminderSwitch.setChecked(AppearancePreferences.isReminderEnabled(this));
        reminderSwitch.setOnCheckedChangeListener((button, checked) ->
                AppearancePreferences.setReminderEnabled(this, checked)
        );
    }
```

代码 34 设置页交互（原文件共 123 行）

代码说明：设置页把单选主题、字号按钮、提醒开关、个人中心和关于入口连接到持久化偏好的管理逻辑。

论文写作提示：说明主题和字号不是临时改变，而是在重启后也能恢复。

第 4.10 节 管理端扩展

管理端复用同一 SQLite 数据库，补充座位维护和预约记录查询。

座位维护

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\activity\AdminSeatActivity.java
范围	第 173 至 260 行
内容	AdminSeatActivity.showSeatDialog

```
    private void showSeatDialog(Seat existing) {
        if (selectedRoom == null) {
            return;
        }
        View content = LayoutInflater.from(this).inflate(
                R.layout.dialog_edit_seat,
```

```
                null,
                false
        );
        TextInputLayout seatNoLayout = content.findViewById(R.id.layout_admin_seat_no);
        TextInputLayout areaLayout = content.findViewById(R.id.layout_admin_seat_area);
        EditText seatNoInput = content.findViewById(R.id.input_admin_seat_no);
        EditText areaInput = content.findViewById(R.id.input_admin_seat_area);
        MaterialAutoCompleteTextView noiseInput = content.findViewById(
                R.id.input_admin_seat_noise
        );
        CheckBox windowCheck = content.findViewById(R.id.check_admin_seat_window);
        CheckBox socketCheck = content.findViewById(R.id.check_admin_seat_socket);

        String[] noiseLabels = getResources().getStringArray(R.array.seat_noise_labels);
        noiseInput.setAdapter(new ArrayAdapter<>(
                this,
                android.R.layout.simple_dropdown_item_1line,
                noiseLabels
        ));
        int defaultNoise = existing == null ? 2 : existing.getNoiseLevel();
        noiseInput.setText(noiseLabels[Math.max(0, Math.min(2, defaultNoise - 1))], false);

        if (existing != null) {
            seatNoInput.setText(existing.getSeatNo());
            areaInput.setText(existing.getArea());
            windowCheck.setChecked(existing.isWindowSeat());
            socketCheck.setChecked(existing.hasSocket());
        }

        AlertDialog dialog = new MaterialAlertDialogBuilder(this)
                .setTitle(existing == null
                        ? R.string.admin_add_seat
                        : R.string.admin_edit_seat)
                .setView(content)
                .setNegativeButton(R.string.action_cancel, null)
                .setPositiveButton(R.string.action_save, null)
                .create();
        dialog.setOnShowListener(ignored ->
                dialog.getButton(AlertDialog.BUTTON_POSITIVE).setOnClickListener(view -> {
                    seatNoLayout.setError(null);
                    areaLayout.setError(null);
                    String seatNo = textOf(seatNoInput).trim();
                    String area = textOf(areaInput).trim();
                    if (seatNo.isEmpty()) {
                        seatNoLayout.setError(getString(R.string.seat_number_required));
                        return;
                    }
                    if (area.isEmpty()) {
                        areaLayout.setError(getString(R.string.seat_area_required));
                        return;
                    }
                    int noiseLevel = noiseIndex(noiseInput.getText().toString(), noiseLabels) + 1;
                    boolean saved;
                    if (existing == null) {
                        saved = seatDao.insertSeat(
                                selectedRoom.getRoomId(),
                                seatNo,
                                area,
                                windowCheck.isChecked(),
                                socketCheck.isChecked(),
                                noiseLevel
                        ) > 0;
                    } else {
                        saved = seatDao.updateSeat(
                                existing.getSeatId(),
                                seatNo,
                                area,
                                windowCheck.isChecked(),
                                socketCheck.isChecked(),
                                noiseLevel
                        );
                    }
                    if (!saved) {
                        seatNoLayout.setError(getString(R.string.admin_seat_save_conflict));
                        return;
```

```
                }
                dialog.dismiss();
                loadSeats();
            })
        );
        dialog.show();
    }
```

代码 35 座位维护（原文件共 274 行）

代码说明：新增和编辑座位共用同一对话框，校验编号、区域、靠窗、插座和噪声等级；座位编号冲突或保存失败时保留对话框并显示错误。

论文写作提示：可以说明管理功能如何复用数据库约束，而不是在每个页面重复验证。

预约记录筛选

文件	D:\android project\ZhiXiPro\app\src\main\java\com\example\zhixipro\db\AdminDao.java
范围	第 69 至 102 行
内容	AdminDao.getReservations

```
public List<AdminReservation> getReservations(String statusFilter) {
    List<AdminReservation> records = new ArrayList<>();
    String where = statusFilter == null || statusFilter.isEmpty()
            ? ""
            : " WHERE r.status = ?";
    String[] args = statusFilter == null || statusFilter.isEmpty()
            ? null
            : new String[]{statusFilter};
    String sql = "SELECT r.reservation_id, u.username, u.display_name, " +
            "rooms.name AS room_name, s.seat_no, s.area, r.start_at, r.end_at, " +
            "r.status FROM " + DatabaseHelper.TABLE_RESERVATIONS + " r " +
            "JOIN " + DatabaseHelper.TABLE_USERS + " u ON u.user_id = r.user_id " +
            "JOIN " + DatabaseHelper.TABLE_SEATS + " s ON s.seat_id = r.seat_id " +
            "JOIN " + DatabaseHelper.TABLE_STUDY_ROOMS +
            " rooms ON rooms.room_id = s.room_id" +
            where +
            " ORDER BY r.start_at DESC";
    try (Cursor cursor = databaseHelper.getReadableDatabase().rawQuery(sql, args)) {
        while (cursor.moveToNext()) {
            records.add(new AdminReservation(
                    cursor.getLong(0),
                    cursor.getString(1),
                    cursor.getString(2),
                    cursor.getString(3),
                    cursor.getString(4),
                    cursor.getString(5),
                    cursor.getLong(6),
                    cursor.getLong(7),
                    cursor.getString(8)
            ));
        }
    }
    return records;
}
```

代码 36 预约记录筛选（原文件共 109 行）

代码说明：管理端把预约、用户、座位和自习室连接查询，支持按状态筛选，并按开始时间倒序显示历史记录。

论文写作提示：与管理员查看爽约记录的界面截图一起使用。

附录 关键布局控件位置

以下文件用于证明课程要求的控件确实被使用。论文正文不必粘贴完整 XML，保留根布局、关键列表控件和运行截图即可。

要求	完整资源路径	位置

LinearLayout	D:\ZhiXiPro\src\mainres\layout\activity_login.xml	根布局第 2 行
RelativeLayout	D:\ZhiXiPro\src\mainres\layout\activity_booking_confirm.xml	根布局第 2 行
ConstraintLayout	D:\ZhiXiPro\src\mainres\layout\activity_home.xml	根布局第 2 行
ListView	D:\ZhiXiPro\src\mainres\layout\activity_study_room_list.xml	第 100 行
RecyclerView	D:\ZhiXiPro\src\mainres\layout\activity_my_booking.xml	第 74 行
RecyclerView 网格	D:\ZhiXiPro\src\mainres\layout\activity_study_calendar.xml	第 152 行
GridView	D:\ZhiXiPro\src\mainres\layout\activity_seat_selection.xml	第 215 行
自绘图表	D:\ZhiXiPro\src\mainres\layout\activity_study_statistics.xml	第 181 行

附录 论文使用建议

- 正文每段代码建议控制在 10 至 30 行，完整方法可以放入附录，避免占用报告主要篇幅。
- 代码截图应在 Android Studio 的 Project 视图中进行，保留文件标签、行号和完整方法名。
- 每段代码后至少说明三个问题：为什么需要它、它解决了什么问题、如何验证它确实有效。
- 测试章节优先展示预约冲突、相邻时段、签到时间窗口、未来预约不误判爽约和重复保存防护。
- RecyclerView 单独作为报告第 5 章，至少说明 Adapter、ViewHolder、LayoutManager、数据刷新和点击事件。